UAV Recovery Firmware Reversing Workbook

LCM-16 firmware image analysis and token recovery

Purpose. Record a reproducible investigation of uav_recovery.fw. This workbook separates container facts verified from the firmware from the candidate acceptance result reported by an earlier custom emulator. Add your own screenshots in the spaces and note the actual output from your Ubuntu run.

Current result status

Working candidate from the supplied analysis notes:

Intern_Pro_Max{r3v3rs3_th3_lcm16_v1rtu4l_c0r3}

The container format, section decoding rules, and CRCs have been independently checked against the firmware. The LCM-16 execution result has not been independently reproduced in this workbook. Treat the candidate as provisional until the bytecode/interpreter check is repeated or otherwise verified.

Challenge request

README_player.txt asks for the authentication token accepted on-device in the form Intern_Pro_Max{...}. It warns that leftover flash strings may be decoys. Therefore, a printable token found in a data section is only a lead; the program logic must determine which token passes.

Step 1 - Inspect the image header

From the challenge directory, first confirm the file and inspect its first bytes. The magic identifies the custom LCMF container; the header contains version, device ID, and section count.

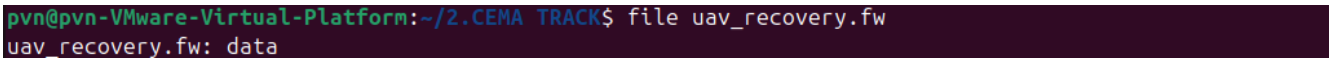
```
cd "$HOME/Desktop/eastern-ctf/2.CEMA TRACK"
ls -l README_player.txt uav_recovery.fw
file uav_recovery.fw
xxd -g1 -l 96 uav_recovery.fw
```

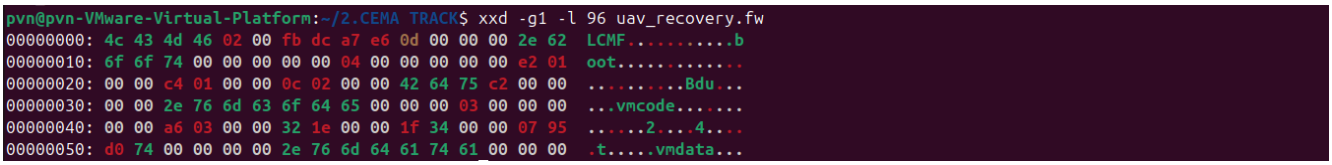
Verified from the provided file: magic LCMF, version 2, device ID 0xe6a7dcfb, and 13 sections. The section table begins at byte 14 and has 36-byte entries; the table ends at byte 482, the start of .boot.

Screenshot 1 - file header

Evidence screenshots are inserted below. They show the file identification and LCMF header inspection.

Caption: File identification and LCMF header inspection.





Step 2 - Parse and validate sections

The .boot bytes are XZ/LZMA-compressed loader notes. Those notes describe flags: 0x1 means zlib, 0x2 means XOR with an xorshift32 stream, and 0x4 means LZMA. The XOR seed is device_id XOR FNV-1a(section name). A section is decoded by undoing XOR first, then decompressing when its compression flag is set. CRC-32 and uncompressed size provide an integrity check.

Create a small inspection script. It does not execute the VM; it only parses, decodes, and checks the container sections.

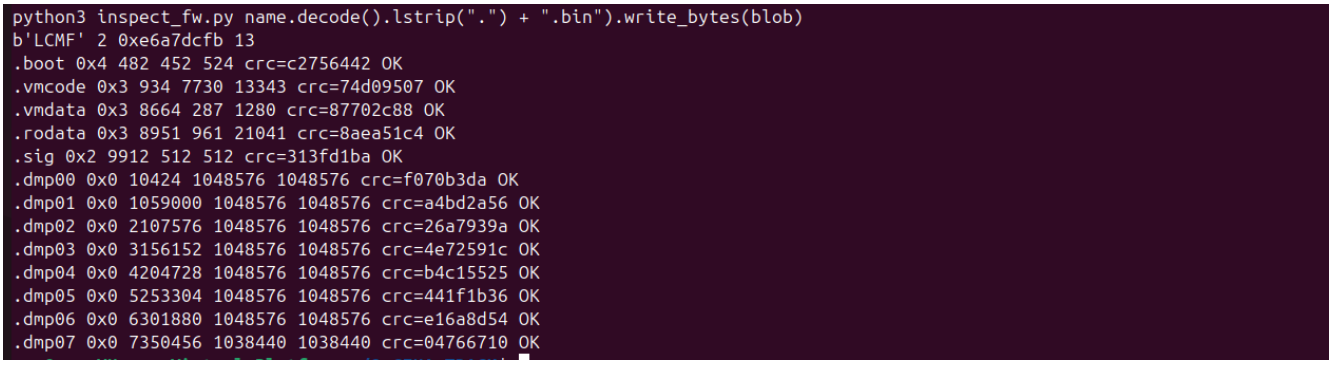
```
cat > inspect_fw.py <<'PY'
from pathlib import Path
import struct, zlib, lzma
image = Path("uav_recovery.fw").read_bytes()
magic, version, device_id, count = struct.unpack_from("<4sHII", image, 0)
assert magic == b"LCMF"
print(magic, version, hex(device_id), count)
def fnv1a(data):
h = 2166136261
for b in data:
h = ((h ^ b) * 16777619) & 0xffffffff
return h
def xs32(x):
x ^= (x << 13) & 0xffffffff
x ^= x >> 17
x ^= (x << 5) & 0xffffffff
return x & 0xffffffff
def xor_stream(blob, name):
state = device_id ^ fnv1a(name)
key = bytearray()
while len(key) < len(blob):
state = xs32(state)
key.extend(state.to_bytes(4, "little"))
return bytes(a ^ b for a, b in zip(blob, key))
for i in range(count):
row = 14 + 36 * i
name, flags, vaddr, offset, packed, raw_len, crc, reserved = \
struct.unpack_from("<10sH6I", image, row)
name = name.split(b"\0", 1)[0]
blob = image[offset:offset + packed]
if flags & 0x2:
blob = xor_stream(blob, name)
if flags & 0x4:
blob = lzma.decompress(blob)
elif flags & 0x1:
blob = zlib.decompress(blob)
good = len(blob) == raw_len and zlib.crc32(blob) == crc
print(name.decode(), hex(flags), offset, packed, raw_len,
f"crc={zlib.crc32(blob):08x}", "OK" if good else "FAIL")
assert good
Path("decoded_" + name.decode().lstrip(".") + ".bin").write_bytes(blob)
```

```
PY
python3 inspect_fw.py
```

Expected check from the inspected image: all 13 section CRCs match. Important decoded artifacts are decoded_boot.bin, decoded_vmcode.bin, decoded_vmdata.bin, decoded_rodata.bin, and decoded_sig.bin. This confirms the extraction and integrity checks; it does not prove that a token is accepted by the VM.

Screenshot 2 - section table and CRCs

Evidence screenshot is inserted below. It shows the section decode and CRC results.



Caption: Section table and CRC verification.

Step 3 - Read loader notes and inspect decoys

The decoded boot section is a compact format reference. Read it and confirm that its flag descriptions match the decoder used in Step 2.

```
strings -a decoded_boot.bin
```

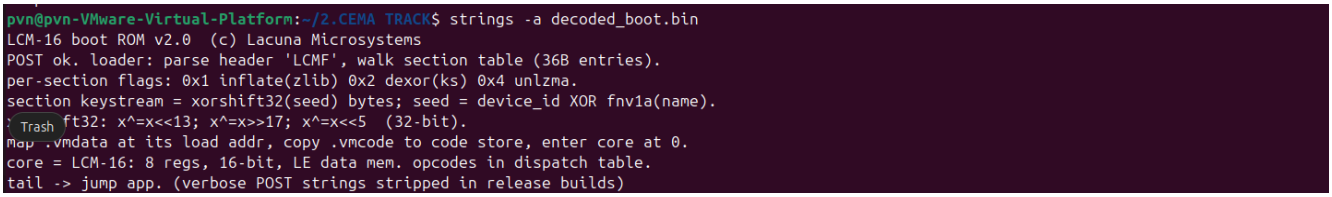
The decoded .rodata contains labeled examples such as last_token, otp.blob, xkey/scratch, and revbuf. The README warns that manufacturing and telemetry leftovers are not necessarily the accepted token. Do not submit a candidate only because it appears in a string table.

```
strings -a decoded_rodata.bin | grep -Ei "token|otp|xkey|scratch|revbuf|Intern_Pro_Max"
```

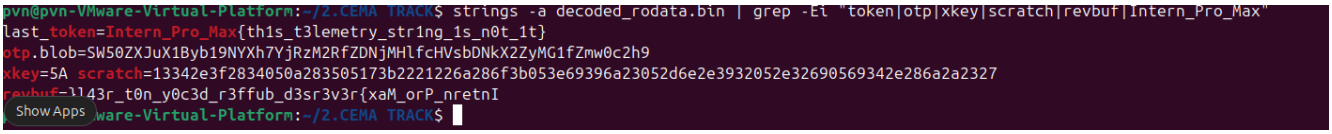
Decision: treat the printable strings as decoys until the program's comparison path confirms an input. The next task is to understand the VM bytecode and its acceptance condition.

Screenshot 3 - loader rules and decoy strings

Evidence screenshots are inserted below. They show the loader rules and labeled decoy strings; these are not proof that a candidate is accepted.



Caption: Loader rules and labeled decoy strings.



Step 4 - Map the LCM-16 program

The boot notes identify a custom 16-bit little-endian core with eight registers and a dispatch table. There is no standard disassembler for this format in the supplied files. A defensible reverse-engineering record should show the opcode decoder, instruction lengths, branch targets, and the evidence that the control-flow walk is complete.

Prior analysis notes report that recursive-descent disassembly covered all 13,343 bytes and that jumps landed on instruction boundaries. They also report opaque-predicate junk blocks (xor r5,r5 followed by jz). These claims should be reproduced from your disassembler before describing them as verified.

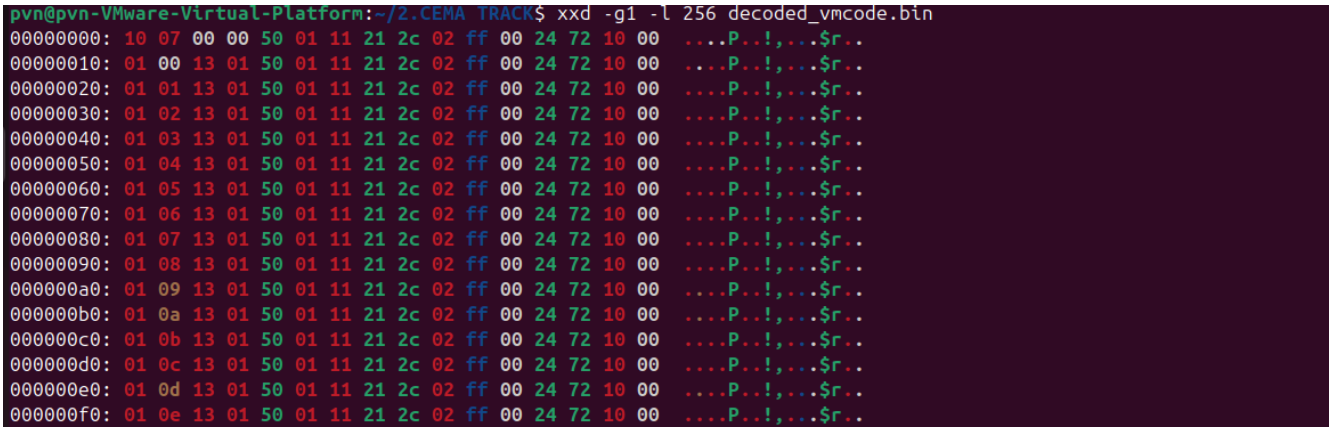
```
xxd -g1 -l 256 decoded_vmcode.bin
xxd -g1 -s 0x100 -l 128 decoded_vmdata.bin
```

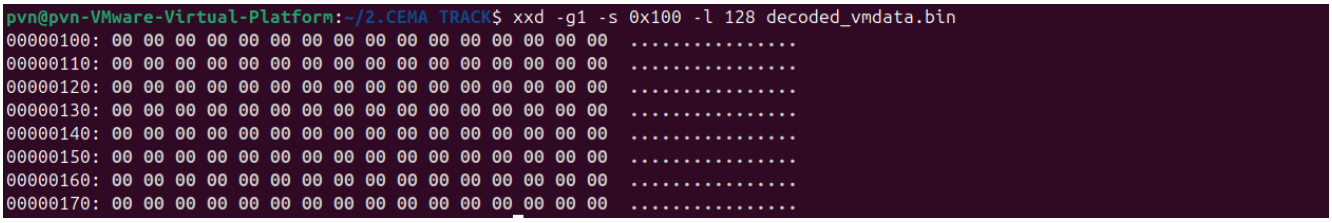
The claimed input path reads 46 characters to memory 0x100..0x12d, checks the prefix Intern_Pro_Max{, the closing brace, and EOF, then processes 30 inner characters. The S-box is reported at VM data address 0x400. Record exact opcode names, operand widths, and branch offsets from the interpreter/disassembler you build.

Screenshot 4 - bytecode and disassembly

The screenshots below show bytecode and VM data hex views. They do not establish a complete disassembly or instruction-coverage result; add that evidence only if you reproduce it.

Caption: Bytecode and VM data hex views; full disassembly remains unverified.





Step 5 - Recover and verify the input

Prior analysis notes describe five rounds over the 30 inner characters: per-position XOR, substitution through the 256-byte S-box, and a running-add chain. The final values are compared with constants, with any mismatch accumulated before output. The notes further report reverse symbolic replay of 445 buffer-store operations, trying byte values to invert each store.

Those details explain the reported candidate, but the current evidence set does not include the VM source, opcode map, or an independently reproducible emulator. The earlier transcript says a hand-built emulator returned output 1 and zero error accumulators for the candidate, and rejected four decoys. Since that emulator was custom-built and is not attached here, record this as a reported result until you run your own implementation or verify the same logic independently.

Candidate to test:

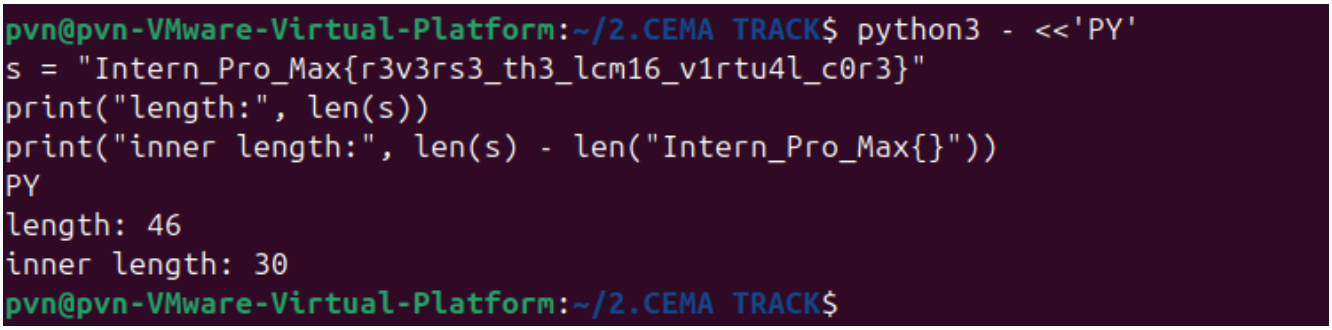
Intern_Pro_Max{r3v3rs3_th3_lcm16_v1rtu4l_c0r3}

```
python3 - <<'PY'
s = "Intern_Pro_Max{r3v3rs3_th3_lcm16_v1rtu4l_c0r3}"
print("length:", len(s))
print("inner length:", len(s) - len("Intern_Pro_Max{}"))
PY
```

This length check validates only the stated input framing (46 total characters and 30 inner characters). It does not execute the firmware. A strong final verification is an independently derived VM run that accepts the candidate and rejects decoys, with the relevant output and input shown together.

Screenshot 5 - independent VM check

The screenshot below shows a length check only. It does not execute the VM. Add your own acceptance and rejected-decoy output after independently reproducing that test.



Caption: Candidate length check only; VM acceptance remains unverified.

Findings and limits

Container findings verified from the supplied firmware: the LCMF version-2 header gives device ID 0xe6a7dcfb and 13 table entries; the 36-byte section table ends at offset 482; the boot notes decode successfully; and the section decoder reproduces every declared uncompressed size and CRC-32.

Candidate from prior reverse-engineering notes: Intern_Pro_Max{r3v3rs3_th3_lcm16_v1rtu4l_c0r3}. The candidate matches the challenge prefix and length. The independent bytecode acceptance test is not established by the container and CRC checks alone.

What the evidence supports

- The firmware container was parsed with the expected entry size and section offsets.

- The boot description matches the decryption/decompression procedure used to recover the program sections.

- All section CRCs and lengths matched the header values.

- Printable .rodata tokens are explicitly not sufficient; the acceptance logic is in the program.

- The final token should be described as confirmed only after a reproducible LCM-16 execution or equivalent bytecode proof.

Personal lab notes

Write only what the screenshots and your own run support. This workbook deliberately leaves room for evidence rather than inventing command output or claiming an independent emulator run that is not shown.

First approach and why it was insufficient

_____

_____

Most useful reverse-engineering clue

_____

_____

How the final check was independently validated

_____

_____

Any correction made to an earlier AI-assisted hypothesis

_____

_____